Escriba la reducción-β de la operación lógica NOT

1. **Elementos base:**
   - TRUE $\equiv \lambda x.\lambda y.x$
   - FALSE $\equiv \lambda x.\lambda y.y$
   - NOT $\equiv \lambda b.b$ FALSE TRUE
2. **Reducir** NOT TRUE**:**
   - **1:** Sustituir la entrada

     $(\lambda b.b$ FALSE TRUE) TRUE
   - **2:** Reemplazar b por TRUE

     $\rightarrow_\beta$ TRUE FALSE TRUE
   - **3:** Expandir TRUE:

     $(\lambda x.\lambda y.x)$FALSE TRUE
   - **4:** Evaluar $x =$ FALSE

     $\rightarrow_\beta$ $(\lambda y.$FALSE)TRUE
   - **5:** Evaluar $y =$ TRUE:

     $\rightarrow_\beta$ FALSE
3. **Reducir** NOT FALSE**:**
   - **1:** Sustituir la entrada

     $(\lambda b.b$FALSETRUE)FALSE
   - **2:** Reemplazar b por FALSE:

     $\rightarrow_\beta$ FALSE FALSE TRUE
   - **3:** Expandir FALSE:

     $(\lambda x.\lambda y.y)$ FALSE TRUE
   - **4:** Evaluar $x =$ FALSE:

     $\rightarrow_\beta$ $(\lambda y.y)$TRUE
   - **5:** Evaluar $y =$ TRUE:

     $\rightarrow_\beta$ TRUE

Escriba y explique como se vería la recursión y los ciclos

En el cálculo lambda puro, como todas las funciones son anónimas y carecen de un identificador persistente en su propio cuerpo, la recursión directa es sintácticamente imposible. Para lograrla se recurre a la autorreferencia mediante combinadores de punto fijo. En lugar de llamarse a sí misma por su nombre, la función se parametriza recibiendo un argumento adicional que representa el "siguiente paso" recursivo, y es el combinador el que se encarga de replicar dinámicamente este término ($Y\ F \rightarrow \beta\ F\ (Y\ F)$), desplegando una nueva capa de computación en cada reducción hasta alcanzar la condición base.

Explique cuando es prudente usar este tipo de programación y cuando no. De un ejemplo para cada caso

Es prudente utilizar este paradigma en entornos de alta concurrencia, procesamiento distribuido y demostración formal de software debido a la inmutabilidad y a la ausencia de efectos secundarios que evitan condiciones de carrera, mientras que no es recomendable en desarrollo de sistemas embebidos, controladores de dispositivos o motores gráficos 3D en tiempo real, donde la asignación constante de marcos de memoria y la recolección de basura provocan latencias frente a la manipulación directa y mutable de bloques de memoria contiguos en lenguajes como C o C++.